

Design and Evaluation of a Deterministic Query Orchestrator for a Distributed Hybrid Retrieval System

Msc Computer Science



**UNIVERSITY
OF LONDON**

Software Project Report
Devang Chauhan

XXXX Words

Excluding title page, abstract, table of content, tables, references and appendix

Abstract

Placeholder

Table of Contents

Abstract	2
Table of Contents	3
List of Figures	5
List of Tables.....	5
1. Introduction.....	6
1.1 Problem Statement.....	6
1.2 Contribution	7
1.3 Aim and Objectives.....	7
2. Literature Review	9
2.1 RAG.....	9
2.2 NL2SQL.....	9
2.3 Resource Selection	10
2.4 Vector Search.....	10
2.5 Relational Search	10
2.6 Distributed Systems	11
2.7 Query Planning.....	11
2.8 Synthesis and Research Gap.....	11
3. Methodology	13
3.1 Scope	13
3.2 Database	14
3.2.1 Schema	15
4. Results and Analysis	18

5. Evaluation.....	19
6. Conclusion.....	20
References	21
Appendices.....	22

List of Figures

Figure - 1 Project Scope.....	14
Figure - 2 Relational Database Schema	16
Figure - 3 Vector Database Schema	17

List of Tables

Table - 2: Sample placeholder	5
-------------------------------------	---

Table - 2: Sample placeholder

1. Introduction

This report begins by establishing the way this project aims to contribute to addressing the challenges involved in the enterprise retrieval systems. It reviews the related work and then moves on to explain the methodology adopted in the design of the orchestrator, followed by the results recorded in the use of the orchestrator against a gold standard benchmark, and lastly, the evaluation and discussion of those results.

1.1 Problem Statement

The advent of large language models (LLMs) [1] has opened up numerous applications where a conversation can be had with an LLM using the knowledge encoded in the parameters of the model during pre-training. While fascinating, this has a limitation when it comes to their use within enterprises. Enterprise data is often gated and hence is not included in the pre-training of the LLMs. This can lead to hallucinations where the LLM confidently responds to queries with false information where it does not have innate knowledge. Retrieval augmented generation (RAG) [2], while based on techniques long known in the domains of search and recommendation systems, has been found to be a reliable way to ground the responses of LLMs in relevant and recent information to reduce hallucinations [3].

RAG is extensively used to provide gated enterprise knowledge to LLMs to ground their responses. Depending on the shape of information, the enterprise knowledge is often distributed across several knowledge stores such as vector stores, document stores, relational tables and graph databases. As this knowledge grows, the knowledge retrieval systems start taking the form of a distributed system where these knowledge stores need to be either partitioned or replicated or both depending on whether consistency or availability is targeted [4].

Also, in enterprise environments, deterministic orchestration is often preferred because retrieval decisions are predictable and auditable, allowing system designers to understand why particular data sources were queried for a given request. However, the performance implications of adopting deterministic orchestration strategies in distributed hybrid retrieval systems remain insufficiently understood.

Little work evaluates how different deterministic orchestration strategies influence end-to-end retrieval latency and retrieval quality when semantic and structured retrieval must be coordinated across distributed knowledge stores. Consequently, practitioners have limited evidence regarding which orchestration strategies provide

the best trade-off between latency and retrieval quality under different sharding configurations.

Lastly, in any information retrieval system, it is important to maintain a balance between the speed and accuracy of the response served to the user to maintain a higher degree of user delight [5]. This project addresses these concerns through its specific contribution.

1.2 Contribution

This project contributes the design and implementation of a deterministic query orchestrator and an experimental comparison of routing and execution strategies within a common sharded retrieval system. It evaluates how these strategies affect retrieval latency and effectiveness across defined workloads and sharding configurations. Here, deterministic strategies follow explicitly defined routing and coordination rules for a given input and system state.

In the information-retrieval literature, hybrid retrieval [6] commonly refers to combining sparse lexical and dense semantic retrieval signals into a unified ranking. The system developed in this project does not perform score or rank fusion. Instead, it is a query orchestrator for a hybrid retrieval system (HRS) supporting structured relational, dense semantic, and sparse lexical retrieval over aligned distributed data through unified orchestration. Here, aligned data refers to corresponding information represented across the different retrieval stores.

1.3 Aim and Objectives

The aim of this project is to design and implement a deterministic query orchestrator for a distributed HRS, and experimentally evaluate how routing and execution strategies affect retrieval latency and effectiveness across defined workloads and sharding configurations.

To achieve this aim, the project has the following objectives:

1. Design and implement a query orchestration framework exposed through a stateless hypertext transfer protocol (HTTP) application programming interface (API) supporting structured relational, dense semantic, and sparse lexical retrieval over aligned distributed data.
2. Implement and compare deterministic shard-routing strategies, including broadcast and selective shard querying, and execution strategies, including filter-then-search and search-then-filter, for applicable query types.

3. Establish a reproducible evaluation benchmark comprising development and held-out test queries, with documented ground-truth results and relevance judgements appropriate to each query type.
4. Evaluate the strategies under controlled workloads and sharding configurations using mean, P95 and P99 retrieval latency [7], precision, recall and F1-score [5], and NDCG@k for ranked results [8]. For known-item queries, measure success as the proportion of queries for which the designated target appears within the top k results.
5. Analyse the trade-offs between latency and retrieval effectiveness, identifying the workload and configuration conditions under which each strategy offers advantages, and explaining the limitations of the findings.

The sections following literary review justify the methods selected to address these objectives, describe their implementation, and evaluate the extent to which the objectives were achieved.

2. Literature Review

This project lies at the intersection of RAG, natural language to SQL (NL2SQL) [9], federated search, vector search, relational databases, and distributed systems. Consequently, the literature review draws upon contributions from each of these domains to motivate the proposed architecture and identify the research gap.

2.1 RAG

RAG has received considerable attention from the research community. Xu et al. [10] propose a distributed RAG (DRAG) system using a peer-to-peer (P2P) network of knowledge stores and local LLMs. Their healthcare-focused approach introduces a novel algorithm that retrieves information locally before querying neighbouring peers. While the system is distributed, it relies solely on vector embeddings [3] as grounding information for LLMs and therefore does not address an HRS, where structured data must also be queried. Similarly, Guerraoui et al. [11] acknowledge the distributed nature of modern retrieval systems and propose RAGROUTE, a neural network classifier that predicts which knowledge store should be queried. Although this reduces unnecessary communication, the routing model is probabilistic and requires retraining whenever the underlying knowledge stores are updated. Recognising that enterprise retrieval extends beyond embeddings, Yu et al. [12] propose TableRAG, which embeds relational table schemas alongside textual content so that retrieved schema embeddings can guide subsequent SQL queries. While this demonstrates the value of hybrid retrieval, the paper does not discuss schema evolution common in large distributed tables are handled in practice.

2.2 NL2SQL

As structured data becomes an increasingly important component of HRS, the quality of NL2SQL translation has a direct impact on retrieval quality. Existing approaches can broadly be categorised as rule-based, neural and hybrid methods. Shalaan et al. [13] propose Guided SQL (GSQL), which combines rule-based processing, natural language processing and vector embeddings to improve SQL generation. SQLizer adopts a different approach by first constructing a query sketch before iteratively materialising it against the database schema using CoreNLP and Word2Vec [14]. More recently, Liu et al. [9] observe that although rule-based systems provide explainability, they struggle with adaptability and complex schemas, while neural approaches remain constrained by training data. Consequently, the field has shifted towards LLM-based methods and fine-tuning techniques that demonstrate stronger generalisation across complex database schemas [9]. Given this direction, implementing a custom NL2SQL engine would contribute relatively little to the

primary aim of this project, which is the design and evaluation of query orchestration rather than query translation itself.

2.3 Resource Selection

Once queries have been translated, the retrieval system must determine which resources should be searched. Federated search addresses this problem by providing a unified interface through which multiple independent collections can be searched and their results combined [15]. Garba et al. [15] identify CORI [16] and ReDDE [17] as two of the most influential resource selection techniques. Callan et al. [16] use probabilistic inference networks to estimate the likelihood that a collection contains relevant information. Here, CORI relies primarily on term frequency (TF) and inverse document frequency (IDF), limiting its ability to capture semantic similarity. Si et al. [17] improve resource selection by estimating collection sizes and emphasise that reducing communication between distributed collections becomes increasingly important as the number of knowledge stores grows. Although these approaches were developed for keyword-based search rather than an HRS, these studies establish that resource selection remains a fundamental challenge in distributed retrieval.

2.4 Vector Search

Within a selected resource, semantic retrieval identifies relevant items by representing documents and queries as vector embeddings, allowing similarity to be measured using functions such as cosine similarity or Euclidean distance rather than exact keyword matching [4]. However, efficiently searching large vector collections presents another challenge. Flat indexes provide exact retrieval but exhibit linear search complexity, which can become costly as dataset size and query volume increase [4]. Modern vector databases therefore employ approximate nearest neighbour (ANN) indexes such as inverted file index (IVF) and hierarchical navigable small world index (HNSW), which can reduce retrieval latency with index configuration affecting the trade-off between search speed and retrieval quality [4, 18]. More recent approaches, including DiskANN and SPANN, further improve scalability by storing large portions of the index on SSDs while maintaining lightweight routing structures in memory, enabling efficient retrieval over billion-scale datasets [19, 20]. These approaches substantially improve vector search performance but focus on optimising retrieval within individual vector stores rather than across distributed knowledge stores.

2.5 Relational Search

While semantic retrieval addresses unstructured information, enterprise knowledge

also resides within relational databases. Relational systems typically employ B-tree indexes for range queries, hash indexes for equality comparisons, while PostgreSQL also provides GIN indexes for full-text and JSON search [4, 21]. Kleppmann and Riccomini [4] note that although indexes improve retrieval performance, they increase storage requirements and slow write operations because they must be maintained as the underlying data changes. Consequently, index selection represents a trade-off between query performance and maintenance overhead.

2.6 Distributed Systems

As enterprise knowledge continues to grow, a single machine may no longer be sufficient to store or process all data. Distributed database systems therefore introduce concerns such as sharding and replication. The degree of fragmentation directly influences query performance [22], while the choice of sharding strategy depends heavily on workload characteristics. Key-range partitioning is effective when queries exhibit locality, whereas hash-based partitioning generally provides a more balanced distribution of reads and writes [4]. Similar principles apply to vector databases, where semantically similar vectors may be clustered together to reduce the search space during retrieval [23]. Distributed systems also introduce opportunities for parallel query execution; however, Özsu and Valduriez [24] note that reducing response time through parallelism may increase overall execution time because of additional communication overhead. Consequently, computation should occur as close to the data as possible whenever feasible [24].

2.7 Query Planning

These considerations ultimately influence query planning within distributed retrieval systems. Özsu and Valduriez [25] argue that query optimisation should minimise communication costs by reducing the amount of data transferred between nodes. They recommend assigning execution costs to alternative query plans and favour strategies that perform selection as early as possible, thereby reducing intermediate results and avoiding expensive join operations wherever possible [25]. Such observations reinforce the importance of orchestration strategies that intelligently coordinate retrieval across multiple knowledge stores.

2.8 Synthesis and Research Gap

The reviewed literature demonstrates progress in semantic retrieval, distributed RAG, resource selection, relational and vector indexing, and query planning. Together, these contributions show that retrieval performance depends on both the efficiency of individual stores and their coordination across a distributed system. Resource selection motivates querying only relevant collections, distributed systems research

highlights communication overhead, and query planning emphasises reducing intermediate results through early selection. These findings motivate examining both which shards are queried and the order in which filtering and retrieval are performed.

However, the reviewed work does not establish how alternative deterministic orchestration strategies compare within a common sharded system supporting structured relational, dense semantic, and sparse lexical retrieval. This project addresses that gap by comparing broadcast and selective shard querying, alongside filter-then-search and search-then-filter execution where applicable, over aligned distributed data. The evaluation examines the trade-offs between retrieval latency and effectiveness across defined workloads and sharding configurations. The following section describes the development methodology and experimental approach used to implement the system and its evaluation.

3. Methodology

A deterministic query orchestrator was developed to address the aim and objectives of the project. The two governing themes during the development were testability and reproducibility. These themes led to the development of the project in the small increments of data acquisition, schema design, shard construction, relational retrieval, vector retrieval, orchestration, end-to-end integration, and benchmarking. All these increments are covered in the following sub-sections.

3.1 Scope

In real world, an HRS receives natural language queries that may require information from multiple knowledge stores. A decision has to be made as to which knowledge stores should be queried, in what order they should be queried, and how the retrieved results should be combined before returning a response. In distributed environments, these decisions directly influence communication overhead, response latency and retrieval quality.

HRS often encounters complex queries. For example:

"Give us three movie options directed by an academy award-winning director that will motivate us to survive through tough times in life."

Such a query needs to be decomposed into the following sub-queries:

Query 1: *"Find movies directed by Academy Award-winning directors."*

Query 2: *"Find movies whose plot summaries describe inspirational stories."*

In real world enterprise applications, LLMs are increasingly used to generate such sub-queries from a complex user query and to synthesise a final response from the retrieved information [26]. These components, however, are not the focus of this project and are therefore kept outside the scope [Figure - 1].

Instead, this project focuses designing the deterministic orchestration layer as an HTTP API responsible for coordinating retrieval across knowledge stores. In a production enterprise application, an LLM will decompose a natural language query into discreet queries the orchestrator can understand, and will request the orchestrator API to retrieve information from the aligned data in knowledge stores.

A user interface, authentication, data update endpoints, and cloud deployment are also kept outside the scope.

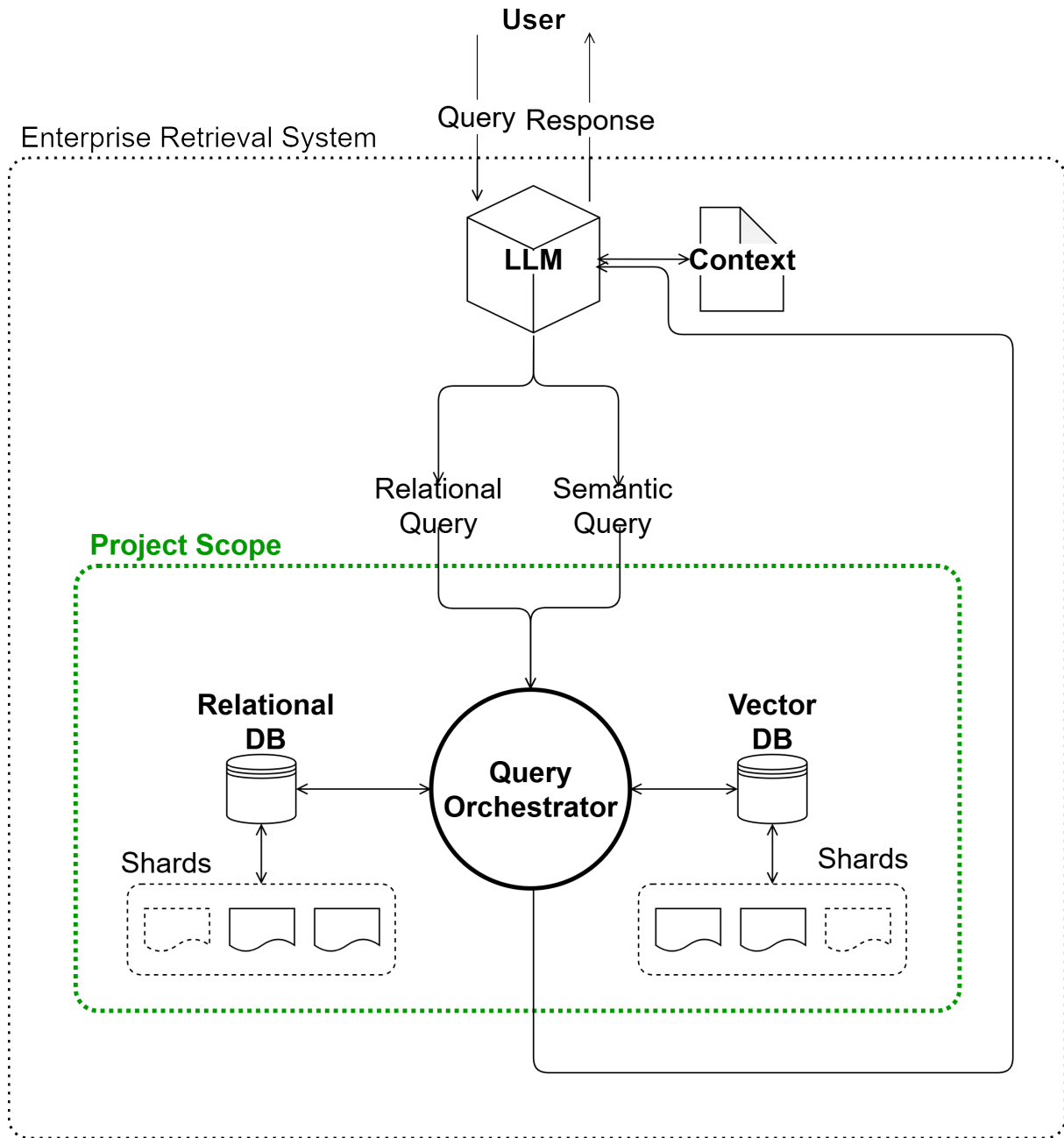


Figure - 1 Project Scope

3.2 Database

To perform evaluation of the orchestrator, it needs to be able to retrieve information from knowledge stores. Since movies contain structured data such as title, release year, director, and descriptive data such as overview, they were selected to build such knowledge stores. Among other alternatives, the movie database (TMDB) [27] was chosen for its free API access to its data, to build the knowledge stores.

PostgreSQL was selected for the relational store because its SQL support, integrity constraints, and transactions suited the structured movie metadata and relationships,

enabling precise filtering and consistent data ingestion. Qdrant [28] was selected for the vector store because it supported dense and sparse retrieval, named vectors, and metadata filtering.

3.2.1 Selection and Preparation

Using TMDB’s discovery filters, two groups were defined: U.S.-origin English-language films and Indian-origin Hindi-language films. These groups were labelled Hollywood and Bollywood, respectively, for this project. Both applied a primary release-date range from 1 January 1990 to 1 August 2026. These specific languages and the time period were chosen for the author’s familiarity with the English and Hindi movies during the period, which was crucial for creating the known-item queries during evaluation.

Movies with a minimum 20 votes were selected in the descending order of popularity, with the exclusion of adult content. A total of 10,000 records were collected from the larger United States/English result set, while all 1,093 records from the India/Hindi group were collected. There were no duplicate movie identifiers between the groups. Records required a non-blank title and overview, resulting in a dataset of 11,093 movies.

Raw responses were cached as JSON before normalisation so that data preparation could be repeated without further API requests to TMDB. From the raw response, the canonical dataset was prepared that retained titles, overviews, release dates, genres, directors, cast, production countries, and a derived industry label. The canonical dataset served as the common input to the Python [29] loader for both the databases.

3.2.2 Layout and Sharding

For the experimental evaluation, two sharding configurations were constructed from the same canonical dataset. The hash layout assigned even movie identifiers to shard A and odd identifiers to shard B, producing 5,485 and 5,608 movies respectively. This provided an approximately balanced distribution, although the assignment did not reflect the content of the movies.

The industry layout placed Hollywood movies on shard A and Bollywood plus other or ambiguous movies on shard B. The resulting shard sizes were 9,559 and 1,534 movies. This configuration provided information that could support selective querying for some industry conditions, but it also introduced an imbalance in collection size. Consequently, the layouts represented different trade-offs between balanced placement and domain-based selection.

3.2.3 Schema

Two complimentary schemas were developed. Figure-2 shows the PostgreSQL tables and their relationships, while figure-3 shows a Qdrant point, its named vectors and its payload. The schemas represent the same movies in forms suited to exact relational conditions and ranked retrieval respectively.

In PostgreSQL, `movies` holds the attributes belonging directly to a movie, including its title, release date, overview and industry. Genres, people and countries occupy separate tables because a movie may have several of each, and each genre, person or country may relate to many movies. The `movie_genres` and `movie_countries` association tables represent these many-to-many relationships; their composite primary keys prevent duplicate associations. The `movie_credits` table links movies to people and retains information specific to a credit, including actor or director role, character name and cast order. Placing those attributes on the credit preserves the distinction between a person and that person's participation in a particular movie.

This relational structure supports explicit predicates over shared entities without repeating their names in every movie record. Foreign keys enforce the relationships within PostgreSQL, while field constraints reject invalid values. The trade-off is that retrieving a complete movie requires joins across several tables. Indexes on the relevant dates, identifiers and relationship fields support those access paths.

Qdrant stores one point per movie within each layout's collection. Each point contains the named vectors `overview_dense`, `full_document_dense` and `metadata_sparse`. Keeping these representations on one point gives them a common identity while allowing their contents to differ. The payload duplicates the fields needed for filtering: `release_year`, `genre_ids`, `industry` and `country_codes`. It also records the movie identifier, layout, shard, embedding version and source/text checksums [30]. This limited duplication allows metadata filtering before performing vector search. PostgreSQL remains the authoritative metadata store.

The same `movie_id` connects both databases. The canonical TMDb identifier is stored as the primary key `movies.movie_id` in PostgreSQL, as the Qdrant point ID, and as `movie_id` in that point's payload. Thus, for a movie with identifier `m`, the relational record and vector point both retain `m` rather than receiving unrelated store-specific identifiers. This avoids matching by title, which may be shared by different movies or vary in spelling, and removes the need for a separate identifier-mapping table. Association-table foreign keys also use this value to connect the movie to its

relational metadata.

3.2.4 Embeddings

The representations separated different sources of retrieval evidence. `overview_dense` embedded the plot overview, allowing semantic matching to be based on descriptive content. `full_document_dense` embedded a deterministic combination of metadata and overview, providing an alternative that also exposes structured attributes to the embedding model. `metadata_sparse` encoded metadata without the overview, making it appropriate for known-item queries containing identifying terms. Keeping the representations distinct made their input content explicit; preparing both dense variants did not itself establish that either was more effective.

Dense embeddings were generated using `text-embedding-3-large` [31] with 1,024 dimensions and compared using cosine distance [32, 33]. Sparse representations were generated using `Qdrant/bm25` [34] through `FastEmbed` [35] as term indices and weights. The dense representations supplied semantic evidence from text, while the sparse representation retained lexical evidence from metadata. These representations were stored under the named-vector fields described in figure - 3.

The model, dimensions and distance function formed a fixed representation contract. Holding them constant allowed strategy comparisons without simultaneously changing the embedding space. The 1,024-dimensional setting and selected model were not independently optimised in this project and should not be interpreted as demonstrated best choices. Dense movie embeddings were generated once, retained and loaded into both layouts, avoiding repeated external generation and ensuring that a layout comparison used the same vectors.

3.2.5 Indexes

PostgreSQL used B-tree indexes on `movies.release_date`, `movies.industry`, `movies.title` and `people.name` to support date-range and equality conditions. Relationship indexes on `movie_genres.genre_id` and `movie_countries.country_code` supported finding movies associated with a particular genre or production country. An index on `movie_credits.movie_id` supported retrieving a movie's credits, while the composite index (`person_id`, `credit_type`) supported finding movies associated with a person in a specified role, such as director. Primary-key indexes also supported identifier-based lookup, including `movies.movie_id` during enrichment. These access paths complemented the normalised schema by supporting conditions and joins across its related tables.

Vector indexes. Qdrant used its default HNSW configuration for dense indexing and an inverted index for the sparse representation. The dense configuration supported similarity search over the 1,024-dimensional embeddings, while the sparse index supported retrieval using term indices and weights. Qdrant's inverse document frequency modifier was enabled for sparse retrieval, so term weighting depended on the contents of each collection. This distinction matters when merging scores from different shards, whose term distributions may differ.

Payload indexes. Qdrant also indexed `release_year` and `genre_ids` as integer fields, and `industry` and `country_codes` as keyword fields. These fields matched the benchmark's structured filters and supported restricting eligible points within the vector store during filter-then-search. Maintaining these access paths required the corresponding metadata to be duplicated consistently from the canonical records into the Qdrant payload.

Index definitions were consistent across layouts and were neither tuned nor evaluated as a separate design variable. They describe the configured access paths, not evidence that every request used a particular index.

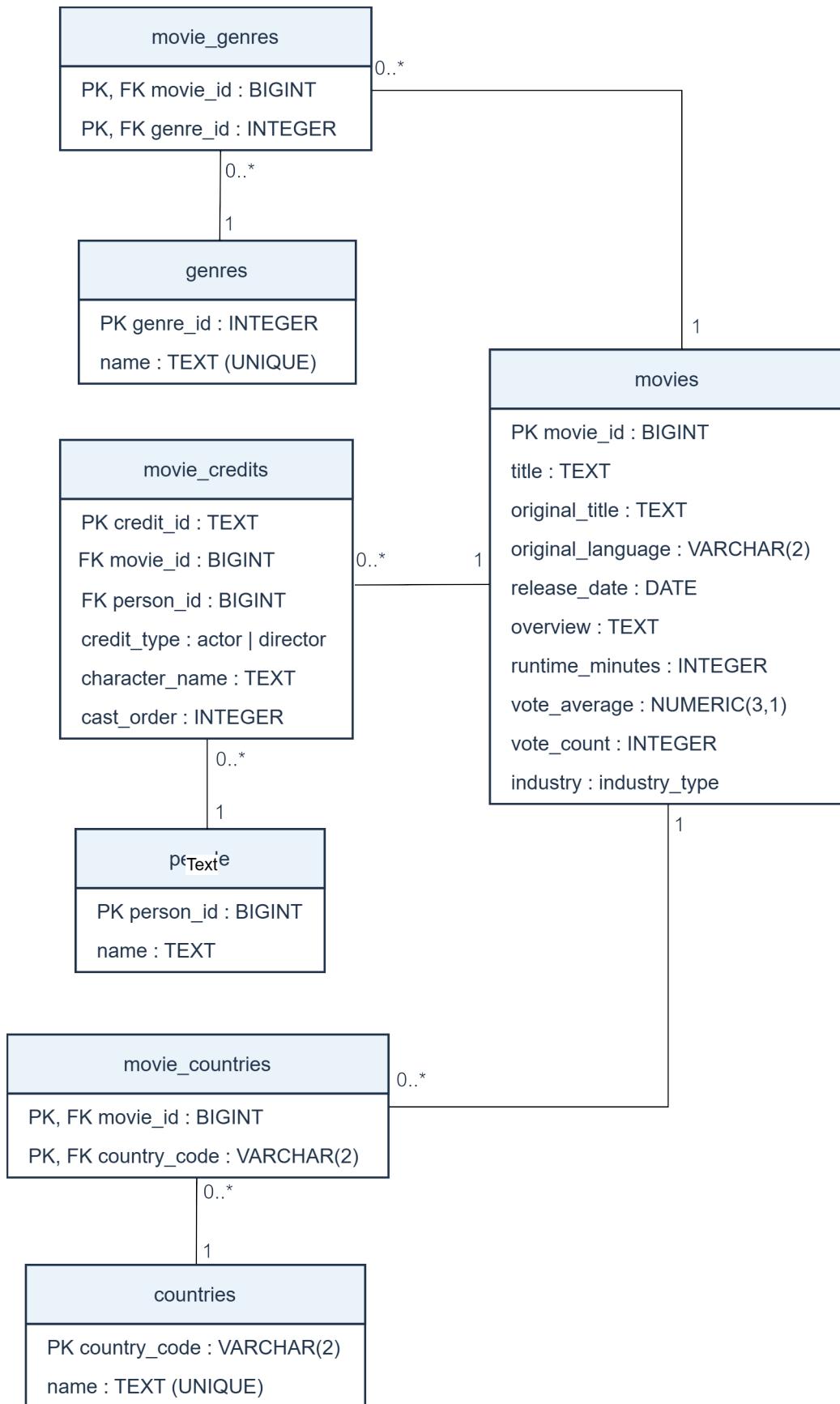


Figure - 2 Relational Database Schema

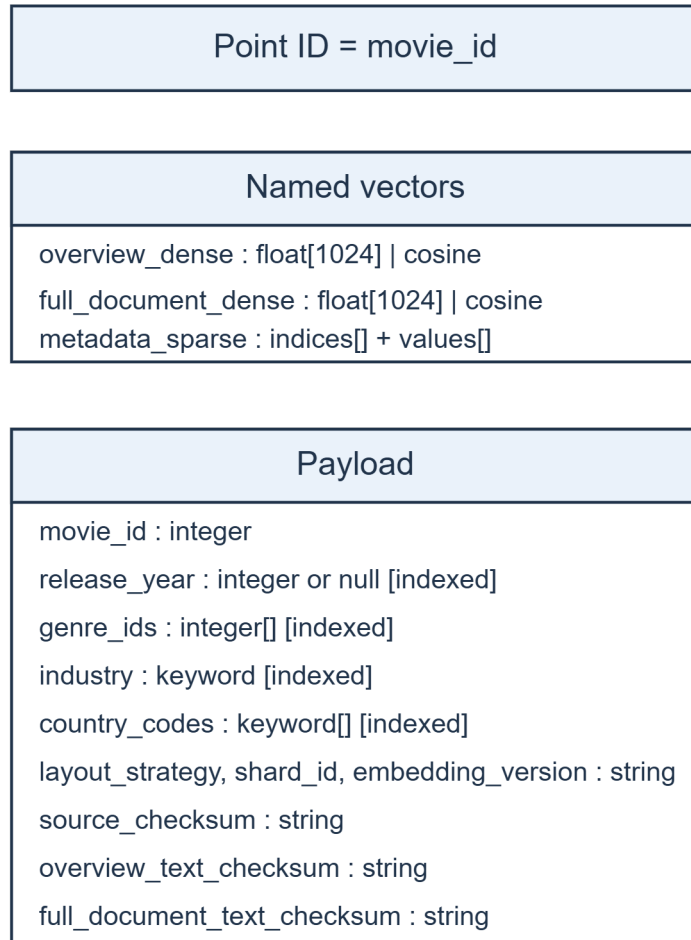


Figure - 3 Vector Database Schema

4. Results and Analysis

Placeholder

5. Evaluation

Placeholder

6. Conclusion

Placeholder

References

Placeholder

Appendices

Placeholder